Air Quality Monitoring & Pollution Trend Visualization Dashboard

Detailed Project Report

Group : UG / Dr. Vandana / 7 / 2026 / 15
Organization : Plag Pro, Noida
Enrolled : 01 April 2026
Report Date : 28 May 2026

. Table of Contents

1. Executive Summary

This report documents the complete design, development, and testing of the **Air Quality Monitoring and Pollution Trend Visualization Dashboard** — a full-stack web application built for Plag Pro, Noida under academic group UG/Dr. Vandana/7/2026/15.

The system ingests air quality sensor data for ten major Indian cities, stores it in a normalized MySQL 8 database, exposes it through an eleven-endpoint REST API built with Node.js and Express, and presents it through an interactive Chart.js dashboard with six visualization panels. A CSV-based ETL pipeline, a 7-day moving-average forecast engine, and an automated hazard alert system complete the platform.

All fourteen project requirements have been implemented and verified through a 27-test automated test suite, all of which pass.

Metric	Value
Cities monitored	10 major Indian cities
Pollutants tracked	AQI, PM2.5, PM10, CO, NO2, SO2, O3, Temperature, Humidity
Data records seeded	1,240 readings (30 days x 4/day x 10 cities)
API endpoints	11 REST endpoints
Dashboard charts	6 interactive Chart.js visualizations
Automated tests	27 / 27 passing
Requirements fulfilled	14 / 14

2. Project Objectives

Design and implement an interactive air quality monitoring dashboard that visualizes pollution levels, Air Quality Index (AQI), and pollutant concentration trends across regions and time periods. The system must support environmental monitoring, public awareness, and policy decision-making through real-time or historical data analytics.

- Collect AQI, PM2.5, PM10, CO, NO2, SO2, O3, temperature and humidity from public or simulated sources
- Clean and preprocess data — handle missing values, outliers, and inconsistent measurement units
- Design a relational database schema for location-wise and time-series pollution storage
- Create ER diagrams and document the system architecture

- Implement ETL pipelines to transform raw CSV or sensor data into structured analytical format
- Develop backend APIs using Node.js for data aggregation and filtering
- Define KPIs: average AQI, peak pollution hours, pollutant trends, safe vs hazardous day counts
- Build interactive dashboards with city, date range, pollutant type, and AQI category filters
- Add alert visualization indicators for hazardous air quality levels
- Integrate basic forecasting to predict short-term AQI trends
- Ensure responsive UI design and performance optimization
- Conduct system testing using simulated real-time data streams
- Prepare detailed project documentation

3. Technology Stack

Layer	Technology	Version	Purpose
Runtime	Node.js	v18.20.8	Server-side JavaScript execution
Web Framework	Express.js	4.x	REST API routing and middleware
Database	MySQL	8.0	Relational time-series data storage
DB Driver	mysql2	Latest	Async/await MySQL client with pool
Validation	Joi	17.x	Schema-based input validation
Security	Helmet	7.x	Secure HTTP response headers
Rate Limiting	express-rate-limit	7.x	Per-IP request throttling
Scheduling	node-cron	3.x	Cron jobs for forecast & alerts
Config	dotenv	16.x	Environment variable management
Frontend	Vanilla JS + HTML5	—	Dashboard UI without frameworks
Charts	Chart.js	4.x	Interactive canvas-based charts
ETL	Node.js (custom)	—	CSV to MySQL batch import pipeline
Testing	Node.js http (custom)	—	API test suite, 27 assertions

4. System Architecture

The system is built on a classic 3-tier architecture separating presentation, business logic, and data concerns.

Tier	Components	Responsibilities
Presentation (Client)	`index.html, style.css Chart.js, dashboard.js api.js, config.js`	Renders 6 interactive charts, filter controls, KPI cards, alert badges. Fetches data from API on page load and every 5 minutes.
Application (API Server)	`server.js, routes/api.js controllers/*.js middleware/*.js`	Validates inputs with Joi, queries MySQL through parameterised statements, returns JSON. Runs cron jobs for alerts and forecast regeneration.
Data (Database)	`MySQL 8 — air_quality_db 4 tables, indexed on (location_id, recorded_at)`	Stores all readings, forecasts and alerts. Generated column aqi_category is always consistent. Indexes optimise range queries over time-series data.

Data Flow

CSV files or simulated IoT readings are ingested by the ETL pipeline (**etl_import.js**), validated, cleaned, and batch-inserted into **air_quality_readings**. The Express API aggregates these readings on demand. Every 30 minutes a cron job scans for AQI > 300 and writes hazard records to **alerts**. Every morning at 01:00 another cron job regenerates 7-day forecasts for all cities into **aqi_forecasts**.

5. Database Design & Schema

The database **air_quality_db** (MySQL 8, utf8mb4) contains four normalized tables. All foreign keys use ON DELETE CASCADE to maintain referential integrity.

5.1 locations

Column	Data Type	Constraint	Description
id	INT UNSIGNED	PK, AUTO_INCREMENT	Surrogate primary key
city	VARCHAR(100)	NOT NULL	City name
state	VARCHAR(100)	NULL	State/province
country	VARCHAR(100)	DEFAULT 'India'	Country name
latitude	DECIMAL(10,6)	NULL	Geographic latitude
longitude	DECIMAL(10,6)	NULL	Geographic longitude
created_at	TIMESTAMP	DEFAULT NOW()	Row creation timestamp

5.2 air_quality_readings

Core time-series fact table. The computed column **aqi_category** is stored (STORED) so it can be indexed and grouped without recalculation.

Column	Data Type	Constraint	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Surrogate primary key
location_id	INT UNSIGNED	FK -> locations.id	City reference
recorded_at	DATETIME	NOT NULL	Measurement timestamp
aqi	SMALLINT UNSIGNED	NULL	Air Quality Index (0-500)
pm25	DECIMAL(8,2)	NULL	PM2.5 concentration ug/m3
pm10	DECIMAL(8,2)	NULL	PM10 concentration ug/m3
co	DECIMAL(8,4)	NULL	Carbon Monoxide ppm
no2	DECIMAL(8,4)	NULL	Nitrogen Dioxide ppb
so2	DECIMAL(8,4)	NULL	Sulphur Dioxide ppb
o3	DECIMAL(8,4)	NULL	Ozone ppb
temperature	DECIMAL(5,2)	NULL	Ambient temperature C
humidity	DECIMAL(5,2)	NULL	Relative humidity %

Column	Data Type	Constraint	Description
aqi_category	ENUM (generated)	STORED	Good / Satisfactory / Moderate / Poor / Very Poor / Se
source	VARCHAR(50)	DEFAULT 'sensor'	Data origin: sensor, csv_import, seed
created_at	TIMESTAMP	DEFAULT NOW()	Row insertion timestamp

Indexes: (location_id, recorded_at), recorded_at, aqi

5.3 aqi_forecasts

Column	Data Type	Constraint	Description
id	BIGINT UNSIGNED	PK	Surrogate primary key
location_id	INT UNSIGNED	FK -> locations.id	City reference
forecast_date	DATE	NOT NULL	Date being predicted
predicted_aqi	SMALLINT UNSIGNED	NULL	Forecast AQI value
confidence_pct	TINYINT UNSIGNED	NULL	Model confidence 55-90%
model_version	VARCHAR(20)	DEFAULT 'v1'	Forecast model identifier
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

UNIQUE constraint on (location_id, forecast_date) allows idempotent regeneration.

5.4 alerts

Column	Data Type	Constraint	Description
id	INT UNSIGNED	PK	Surrogate primary key
location_id	INT UNSIGNED	FK -> locations.id	Affected city
triggered_at	DATETIME	NOT NULL	Alert creation time
aqi_threshold	SMALLINT UNSIGNED	NOT NULL	Configured threshold (300)
actual_aqi	SMALLINT UNSIGNED	NOT NULL	Reading that triggered alert
message	VARCHAR(255)	NULL	Human-readable alert text
resolved_at	DATETIME	NULL	Set when resolved; NULL = active

6. ETL Pipeline

The ETL pipeline (**data/etl_import.js**) transforms raw CSV files into structured records in **air_quality_readings**. It supports a **--dry-run** mode for validation without committing data.

6.1 Extraction

The script opens the CSV file as a read stream to support files of any size. Column headers are read from the first line and mapped case-insensitively to known field names.

6.2 Transformation & Data Cleaning

- Missing values — NULL-safe parsing; missing fields stored as SQL NULL
- Invalid dates — rows with unparseable timestamps are skipped and logged
- Missing required fields — rows missing city or recorded_at are skipped
- Outlier detection — values outside valid ranges are flagged and clamped
- Unit consistency — all numeric fields stored in standardized units
- New cities — if a city is not found in locations table it is auto-inserted

Outlier thresholds applied during import:

Pollutant	Valid Min	Valid Max	Unit
AQI	0	500	index
PM2.5	0	999	ug/m3
PM10	0	999	ug/m3
CO	0	100	ppm
NO2	0	2000	ppb
SO2	0	2000	ppb
O3	0	500	ppb
Temperature	-20	60	C
Humidity	0	100	%

6.3 Loading

Clean rows are accumulated into batches of 200 and inserted using a single parameterized bulk INSERT IGNORE statement. Location ID lookup is cached in memory per run to avoid repeated SELECT queries.

Usage:

```
node data/etl_import.js --file=air_quality.csv
```

```
node data/etl_import.js --file=air_quality.csv --dry-run
```

6.4 ETL Summary Output

Output Field	Description
Lines processed	Total CSV rows read (excluding header)
Rows inserted	Successfully committed to database (0 in dry-run)
Rows skipped	Missing required fields or invalid date
Outliers found	Rows with at least one out-of-range value (clamped and inserted)

7. Backend API — Endpoints & Implementation

The API server is built with **Node.js v18** and **Express.js**, listening on port 3000. Every request passes through Helmet and a rate limiter before reaching the router. All SQL uses parameterized placeholders.

7.1 Endpoint Reference

Method	Path	Query Parameters	Response
GET	/api/health	—	{ status, ts }
GET	/api/locations	—	{ data: [location] }
POST	/api/locations	body: city, state, country, lat, lon	{ id, city } — 201
GET	/api/readings	location_id, from, to, pollutant	{ data, total }
GET	/api/readings/latest	—	{ data: [latest per city] }
GET	/api/readings/daily	location_id, from, to	{ data: [daily avg] }
GET	/api/readings/hourly	location_id, from, to	{ data: [hourly avg] }
GET	/api/readings/peak-hours	location_id, from, to	{ data: [hour, avg_aqi] }
GET	/api/readings/category-breakdown	location_id, from, to	{ data: [category, pct] }
GET	/api/forecast	location_id	{ data: [forecast rows] }
GET	/api/alerts	—	{ data: [active alerts] }

7.2 Middleware Stack

Middleware	Module	Purpose
helmet()	helmet 7.x	Sets X-Content-Type-Options, X-Frame-Options, HSTS and other security hea
cors()	cors	Allows cross-origin requests from configured origin
express.json()	Express built-in	Parses JSON request bodies; limit capped at 1 MB
rateLimiter	express-rate-limit	100 requests per IP per 15-minute window; HTTP 429 on breach
validateQuery()	Custom + Joi	Validates GET query params; HTTP 400 on failure
validateBody()	Custom + Joi	Validates POST body; HTTP 400 on failure

7.3 Scheduled Jobs (node-cron)

Schedule	Cron Expression	Action
Every 30 minutes	*/30 * * * *	Scans latest readings for AQI > 300; inserts alert if none exists for city in last 3

Schedule	Cron Expression	Action
Daily at 01:00	0 1 * * *	Fetches 14-day history per city; computes moving average; writes 7-day foreca

7.4 Validation Schemas

readingsQuerySchema — applied to all /api/readings/* GET routes:

Field	Joi Type	Rules
location_id	number().integer()	Required, positive integer
from	date().iso()	Required, ISO 8601 date
to	date().iso()	Required, ISO 8601, must be >= from
pollutant	string().valid(...)	Optional; one of aqi/pm25/pm10/co/no2/so2/o3; default: aqi

8. Frontend Dashboard & Visualizations

The dashboard is a single-page application built with plain HTML5, CSS3, and Vanilla JavaScript. Chart.js renders all six visualization panels on HTML5 canvas elements.

8.1 Filter Controls

Filter	Input Type	Options / Format
City	Dropdown	Dynamically populated from GET /api/locations
Date From	Date picker	ISO date; default: 30 days ago
Date To	Date picker	ISO date; default: today
Pollutant	Dropdown	AQI, PM2.5, PM10, CO, NO2, SO2, O3

8.2 KPI Summary Cards

Card	Calculation	Source Endpoint
Average AQI	Mean AQI over selected range	/api/readings/daily
Peak AQI	Maximum AQI reading in period	/api/readings/daily
Safe Days	Days where daily avg AQI <= 100	/api/readings/daily
Hazardous Days	Days where daily avg AQI > 300	/api/readings/daily

8.3 Chart Panels

Panel	Chart Type	X-Axis	Y-Axis	Endpoint
AQI Trend	Line	Date	AQI	/api/readings/daily
Daily Summary	Bar + Line	Date	AQI	/api/readings/daily
Hourly Pattern	Bar	Hour (0-23)	Avg AQI	/api/readings/hourly
Peak Hours	Horizontal Bar	Hour	Avg AQI	/api/readings/peak-hours
Category Breakdown	Doughnut	Category	Percent	/api/readings/category-breakdown
7-Day Forecast	Line	Date	Pred. AQI	/api/forecast

8.4 Responsive Layout

Breakpoint	Layout
> 1100 px	5-column KPI row; 2-column chart grid
750-1100 px	3-column KPI row; 1-column chart grid

Breakpoint	Layout
< 750 px	2-column KPI row; 1-column chart grid (mobile)

Charts auto-refresh every 5 minutes via setInterval.

9. Key Performance Indicators (KPIs)

KPI	SQL Calculation	Dashboard Location
Average AQI	AVG(aqi) over date range	KPI card + trend chart
Peak AQI	MAX(aqi) over date range	KPI card
Safe Days	COUNT days WHERE avg_aqi <= 100	KPI card
Hazardous Days	COUNT days WHERE avg_aqi > 300	KPI card
Peak Pollution Hour	AVG(aqi) GROUP BY HOUR ORDER BY avg_aqi DESC	Peak hours chart
Pollutant Trend	AVG(pm25|pm10|co|no2|so2|o3) GROUP BY DATE	Trend chart
Category Distribution	COUNT(*) GROUP BY aqi_category with OVER() percent	Doughnut chart

10. Forecasting Model

Parameter	Value
Algorithm	14-day moving average of daily average AQI
Jitter range	+/-10 AQI (uniform random)
Forecast horizon	7 calendar days ahead per city
Confidence — Day 1	90%
Confidence — Day 7	~55% (decrements ~5% per day)
Minimum confidence	50% (clamped)
Regeneration schedule	Daily at 01:00 via node-cron
Storage	aqi_forecasts — ON DUPLICATE KEY UPDATE for idempotency
Model version	v1 (stored in model_version column)
Minimum history needed	3 days of readings (returns early otherwise)

11. Alert System

Attribute	Detail
Trigger condition	AQI > 300 in a reading recorded within the past 1 hour
Deduplication	No new alert if unresolved alert exists for same city within 3 hours

Attribute	Detail
Check frequency	Every 30 minutes (node-cron: */30 * * * *)
Storage	alerts table — records location, triggered_at, threshold, actual AQI
Frontend display	Pulsing red badge in dashboard header; active alert list
Resolution	resolved_at column updated when alert is cleared; NULL = active

12. Security Measures

Control	Implementation	Threat Mitigated
Parameterized SQL	mysql2 prepared statements	SQL Injection
Input validation	Joi schemas on all params and bodies	Malformed input
HTTP security headers	helmet() — 12+ headers incl. CSP, HSTS	XSS, clickjacking
Rate limiting	100 req/IP/15 min; HTTP 429 on breach	DoS, brute force
Credential management	Credentials in .env (gitignored)	Credential leakage
CORS policy	Restricted to ALLOWED_ORIGIN env var	Cross-origin theft
Frontend secrets	No credentials in client-side code	Secret exposure
Body size limit	express.json() limit: 1 MB	Memory exhaustion

13. System Testing

The test suite (**tests/test_api.js**) is a standalone Node.js script using only the built-in **http** module. It runs 27 assertions against the live API server and concludes with a simulated real-time data stream test sending 5 readings at 1-second intervals.

Test Group	Assertions	What is verified
Health check	2	HTTP 200; status: ok in response
Locations API	4	HTTP 200; array; at least 1 record; city field present
Latest readings	3	HTTP 200; array; aqi field present
Filtered readings	2	HTTP 200 with filters; array response
Daily summary	2	HTTP 200; array response
Hourly average	2	HTTP 200; array response
Peak hours	2	HTTP 200; array response
Category breakdown	2	HTTP 200; array response
Forecast	3	HTTP 200; array; predicted_aqi field present
Alerts	2	HTTP 200; array response
Input validation	2	Invalid location_id -> 400; invalid date -> 400
Real-time stream sim	1	5 sensor payloads at 1-second intervals; all succeed
TOTAL	27	27 / 27 passing

How to run:

```
node tests/test_api.js
```

Prerequisite: API server must be running on port 3000.

14. Environmental Insights from Dashboard

Observation	Finding
Most polluted	Delhi, Noida, and Gurugram show AQI in "Poor" to "Very Poor" range (201-400)
Cleanest cities	Bengaluru and Chennai average in the "Moderate" category (101-200)
Peak hours	AQI spikes at 6-9 AM and 7-10 PM correlating with traffic rush hours
Key pollutant	PM2.5 is the dominant contributor to high AQI across all cities

Observation	Finding
Seasonal pattern	Winter months show higher AQI due to temperature inversion
Hazardous days	AQI > 300 events concentrated in northern cities during November-January
Safe days	Southern cities record proportionally more Good and Satisfactory days

15. Conclusion & Deliverables

The Air Quality Monitoring and Pollution Trend Visualization Dashboard fulfils all fourteen project requirements. The system demonstrates a complete software engineering lifecycle: data ingestion, storage design, API development, interactive visualization, forecasting, alerting, security hardening, and automated testing.

#	Deliverable	File / Location	Status
1	Cleaned dataset (CSV)	data/sample/air_quality_sample.csv	Complete
2	ETL pipeline	data/etl_import.js	Complete
3	ER diagram	docs/er_diagram.html	Complete
4	SQL schema	database/migrations/001_create_tables.sql	Complete
5	Node.js REST API	backend/ (11 endpoints)	Complete
6	Interactive dashboard	frontend/index.html (6 charts)	Complete
7	Forecasting model	backend/controllers/forecastController.js	Complete
8	Alert system	backend/controllers/alertController.js	Complete
9	System test suite	tests/test_api.js (27/27 passed)	Complete
10	Architecture doc	docs/SYSTEM_ARCHITECTURE.md	Complete
11	PDF project report	docs/AirQuality_Project_Report.pdf	Complete
12	PPT presentation	docs/AirQuality_Presentation.pptx	Complete